Qwen

Blueprint for a Compliant AI Art Engine: An Architectural Model for the PromptLang-Lua Plugin

Core System Architecture and Initialization Module Design

The foundational architecture of PromptLang-Lua must be built upon a principle of modularity and clear separation of concerns, enabling both deep integration for critical functions like safety enforcement and cross-platform compatibility as a standalone plugin . This duality necessitates a core system layer that provides a stable, well-defined foundation upon which specialized modules can be built and orchestrated. The primary goal of this layer is to manage the entire lifecycle of the assistant—from initialization and configuration loading to module orchestration and the definition of a standardized public interface for host applications. A recommended starting point for this architecture consists of four to five key files: `main.lua` as the central entry point, `config/default.lua` for baseline settings, `config/schema.lua` for configuration validation, and `lib/utils.lua` for shared utility functions. This structure ensures that the core logic remains decoupled from platform-specific details and user-defined configurations, promoting maintainability and ease of deployment.

The `main.lua` file serves as the heart of the application, acting as the single point of entry for any host chat platform wishing to integrate PromptLang-Lua. Its responsibilities are multifaceted and critical to the system's operation. First, it is tasked with creating and initializing the Lua state, ensuring a clean and predictable runtime environment [1][2]. Following standard best practices for performance and security, this involves selectively opening only the necessary standard Lua libraries rather than loading them all by default [1][2]. For instance, one would explicitly call `luaopen_base` and `luaopen_package` instead of using the blanket `luaL_openlibs` function [1][2]. This minimizes the attack surface and reduces memory footprint, aligning with the project's goal of being a lightweight, platform-agnostic plugin. Second, `main.lua` is responsible for orchestrating the loading of all other modules. It will require and initialize the configuration manager, the safety filter, the generative art modules, and the platform interface layer. This centralized control allows for a logical bootstrapping sequence, where dependencies are

resolved in a predictable order. Finally, and most importantly, `main.lua` must define and expose a standardized Application Programming Interface (API) to the host platform. This API should consist of a few clear, well-documented functions that the parent application can call to interact with the assistant, such as `processInput(chatSessionId, userInput)` and `handleRequest(requestType, parameters)`. By abstracting the complex internal workings behind this simple API, `main.lua` makes PromptLang-Lua highly referenceable and easy to integrate, fulfilling a key requirement of the research goal .

Configuration management is a cornerstone of a flexible and portable system. Rather than hardcoding values like API endpoints, model paths, or safety thresholds, these parameters should be externalized into dedicated configuration files. The proposed structure includes two files for this purpose: `config/default.lua` and `config/schema.lua`. The `default.lua` file will contain all the essential configuration variables with sensible fallback values. For example, it might define `api_endpoints.stable_diffusion = "https://api.stability.ai/v2/generation"` and `safety.thresholds.violence_score = 0.8`. This file provides the baseline behavior of the assistant. Separately, `config/schema.lua` will define the expected format and data types for these configuration options. This schema acts as a contract, ensuring that any user-provided configuration file conforms to the expected structure. While the provided context does not detail a specific schema validation library for Lua, implementing such a feature is a common practice in robust software design to prevent runtime errors caused by malformed configurations. The combination of defaults and a schema creates a resilient configuration system that is both flexible for users who wish to customize the assistant and robust against invalid inputs. This approach mirrors principles seen in enterprise-level applications that require support for messaging, security, and database access through configurable connectors [52].

To maintain code cleanliness and avoid duplicating common functionality across different modules, a shared library of utility functions is essential. The `lib/utils.lua` file is designated for this purpose. This module would house generic helper functions that are too specific for a standard library but too general to be included in domain-specific modules. Examples include string manipulation routines for handling prompts, table iteration helpers for processing structured data, logging functions for debugging and monitoring, and error-handling utilities. For instance, a function to safely parse JSON strings or another to check if a string contains balanced brackets could be placed here. By centralizing these utilities, `lib/utils.lua` promotes code reuse, improves readability, and simplifies maintenance. If a bug is found in a utility function, it needs to be fixed in only one place. This practice is a fundamental tenet of good software engineering and is crucial for managing the complexity of a system with up to 20 files. Furthermore, this file

can encapsulate platform-agnostic logic, such as the implementation of a custom parser, which is discussed in a later section, keeping the core system logic pristine and focused on its primary orchestration duties.

The overall architecture should be designed with the principles of extensibility and maintainability at its core. Drawing inspiration from modern software design patterns, the system should be architected to allow new features to be added without modifying existing core code. For example, if a new type of generative art is desired in the future, it should be possible to add a new module, such as `art/new_style/main.lua`, and have the main orchestrator in `main.lua` automatically discover and load it. This "microservices thinking," applied even within a monolithic script, is a powerful paradigm for managing complexity [38]. Similarly, the modular agent architectures used in multi-agent collaboration systems emphasize reusable servers and independent code integration, a principle that directly applies here [39]. By treating each functional component—be it a generative engine, a safety filter, or a utility—as a distinct, interchangeable unit, the architecture becomes more robust and adaptable. This design philosophy ensures that the development of PromptLang-Lua can scale effectively, allowing for continuous improvement and the addition of new capabilities while maintaining the integrity and stability of the core system.

File/Module	Primary Responsibility	Key Functions
`main.lua`	Entry point and system orchestrator.	Initializes Lua state; loads modules; defines the public API for host platforms. [121153]
`config/default.lua`	Provides default configuration values.	Defines fallback settings for API keys, model paths, safety thresholds, etc. [29]
`config/schema.lua`	Defines the structure for configuration files.	Specifies data types and formats to validate user configurations. [135]
`lib/utils.lua`	Central repository for general-purpose utility functions.	Contains reusable code for string/table manipulation, logging, and error handling.

This foundational layer, composed of these core files, establishes a professional-grade architecture for PromptLang-Lua. It prioritizes modularity, flexibility, and a clean separation of concerns, providing a solid base upon which the more specialized and complex functional modules can be built. This approach directly addresses the user's requirements for a system that is not only powerful and artistic but also robust, secure, and easy to integrate into a wide variety of chat platforms.

Generative Art Engine: Modular Components for Descriptive, Graphical, and Style-Based Outputs

The generative art engine is the creative core of PromptLang-Lua, tasked with translating textual prompts into a diverse range of artistic outputs. To achieve the specified functionalities of descriptive art, pixel-knowledge graphs, and handpainting-inspired responses, a monolithic implementation would be unmanageable and brittle. Instead, the optimal architecture employs a strictly modular design, separating each distinct generative capability into its own dedicated set of files. This approach allows for independent development, testing, and optimization of each component, fostering innovation and maintainability. A proposed structure allocates six to seven files to this engine, organized into three primary functional categories: descriptive text-to-image generation, knowledge graph construction from visual or textual data, and generative painting inspired by human artistic styles. Each category is further broken down into an orchestration module and specialized sub-modules, creating a clean hierarchy that manages complexity without sacrificing performance or functionality.

The first major pillar of the art engine is descriptive art generation, which involves converting natural language descriptions into images. This requires sophisticated prompt engineering and interaction with external text-to-image models. The core of this functionality would reside in the `art/descriptive/main.lua` file, which acts as the orchestrator. This module's responsibility is to receive a raw user prompt, pass it through a refinement pipeline, and then send the optimized prompt to a backend service like Stable Diffusion or Midjourney. The refinement process is critical and would be handled by a dedicated submodule, `art/descriptive/prompt_builder.lua`. This file would contain logic for structuring the prompt according to established best practices, such as defining a directive, assigning a role, providing examples, and specifying an output format [11]. Research into tools like PromptPaint demonstrates the value of interactive refinement, suggesting that this prompt-building module could evolve to support iterative improvements based on user feedback [12] [58]. The `prompt_builder` could also incorporate techniques for generating multiple variations of a prompt to explore different artistic interpretations before sending a final version to the generator. This modular separation of building and executing the prompt allows the underlying refinement strategies to be updated or replaced without altering the core communication logic with the generative backend.

The second pillar, the creation of pixel-knowledge graphs, represents a more advanced form of data representation that bridges the gap between visual information and structured knowledge. This functionality is directly supported by the concept of

knowledge graphs, which are graph structures representing real-world entities and their relationships [1] [5]. Recent advancements in multi-modal knowledge graph (MMKG) construction show how knowledge can be extracted from both images and text [3] [4]. The implementation for this feature would be centered around the `art/knowledge_graph/main.lua` file, which would serve as the entry point for this module. This orchestrator would accept various inputs—such as an image URL, a scene description, or even a link to a document—and route them to the appropriate processing function. The heavy lifting of actually constructing the graph would be delegated to a specialized model loader, housed in `art/knowledge_graph/model_loader.lua`. This file would be responsible for loading pre-trained models, potentially leveraging technologies like Graph Neural Networks (GNNs) for efficient processing of large datasets [90] [125] or vision-language models for open-vocabulary scene graph generation [159]. Research indicates methodologies for constructing knowledge graphs from images using LLMs [62] and detecting visual relationships within scenes to build these graphs [63]. The `model_loader` would manage the interaction with these complex models, feeding them input and returning a structured graph representation (e.g., in JSON or a similar format) that can be passed to the renderer. This separation ensures that the model's complexity is encapsulated, and the rest of the system interacts with a clean, simplified interface.

The third pillar, generating handpainting-inspired responses, focuses on style transfer and procedural generation of artistic elements that mimic human brushwork. This area draws heavily from research in generative adversarial networks (GANs) and diffusion models [87]. Models like Stroke-GAN are designed to generate strokes that closely resemble those of human artists, improving the quality of painted details [14]. Other research introduces novel networks for transferring painting styles to new images, often using diffusion models for fine-grained control [15] [34]. The `art/handpainting` directory would contain the modules for this capability. The main orchestrator, `art/handpainting/main.lua`, would take a source image (or a description of a desired style) and an input subject, and invoke the appropriate style transfer algorithm. A dedicated submodule, `art/handpainting/style_transfer.lua`, would encapsulate the logic for applying learned artistic styles, potentially learning from an artist's individual patterns of brushstrokes and color choices [17]. Alternatively, for more abstract outputs, this module could leverage generative models to create entirely new compositions in a given style. Furthermore, a low-level module for pixel-level manipulation, perhaps called `art/handpainting/procedural_generator.lua`, could be developed to draw images pixel by pixel, a technique related to run-length encoding compression schemes [108]. This granular, style-focused approach allows for a rich and varied output, moving beyond simple text-to-image synthesis into the realm of true digital artistry.

Finally, all generated outputs, whether they are images, graphs, or styled text, must be formatted for presentation within a chat session. This task falls to a bridge module, `art/renderer.lua`. This file acts as the universal translator, taking the structured data produced by the generative modules and converting it into a format suitable for display. For a descriptive art prompt that results in an image, the renderer would generate a Markdown image tag (`![](url)`) or a base64-encoded data URI. For a knowledge graph, it might produce a JSON object that can be rendered as a clickable diagram by the host platform's frontend. For a handpainting-style response, it could generate a textual description accompanied by a stylized ASCII-art representation or a simple SVG graphic. By centralizing the rendering logic, `art/renderer.lua` ensures consistency in how outputs are presented to the user, regardless of which generative module produced them. This final step completes the generative loop, transforming the abstract output of the AI models into tangible, interactive elements that enhance the chat experience, fully realizing the artistic potential envisioned for PromptLang-Lua.

Directory	File	Purpose	Key Technologies/Concepts
`art/descriptive/`	`main.lua`	Orchestrates text-to-image generation requests.	Interaction with external APIs (e.g., Stability AI).
	`prompt_builder.lua`	Refines and structures raw user prompts for better generation.	Prompt engineering principles [11], iterative refinement [12].
`art/knowledge_graph/`	`main.lua`	Entry point for knowledge graph generation tasks.	Input routing, graph representation management.
	`model_loader.lua`	Manages loading and interaction with graph generation models.	Graph Neural Networks (GNNs), Vision-Language Models (VLMs) [159], unsupervised graph generation [2].
`art/handpainting/`	`main.lua`	Orchestrate style transfer and generative painting processes.	Learning artist styles [17], stroke-based GANs [14].
	`style_transfer.lua`	Focuses specifically on applying learned artistic styles.	Diffusion models [15], Generative Adversarial Networks (GANs) [87].
`art/`	`renderer.lua`	Converts generated data into display-friendly formats for chat.	Markdown, JSON, SVG, Base64 encoding.

Safety and Policy Compliance: The "Flood-Routing" Interception Framework

The most critical and non-negotiable aspect of the PromptLang-Lua architecture is its safety and policy compliance framework. The user's specification for a "flood-routing"

mechanism that intercepts destructive or violent descriptions and transforms them into compliant artistic forms places immense architectural demands on this component . This system must function as a vigilant gatekeeper, enforcing strict content policies without stifling creativity. The optimal design for this functionality is a dedicated, multi-layered module that acts as a central interception point for all potentially sensitive requests. This framework should be implemented in a separate directory, such as `safety/`, with a series of interconnected files that handle everything from basic keyword filtering to sophisticated semantic analysis and content transformation. This approach ensures that safety logic is cleanly separated from generative logic, adhering to the principle of single responsibility and making the system's compliance mechanisms transparent and auditable.

The central nervous system of this safety framework is the `safety/filter.lua` module. This file serves as the primary interceptor, positioned between the user's input and the generative art modules. Its workflow is the embodiment of the "flood-routing" concept. When a request comes in—for example, a command to generate an image or a prompt containing emotionally charged language—the filter is the first line of defense. It performs a preliminary check, likely triggered by keywords associated with violence or policy conflicts [106][123]. If a trigger is detected, the filter does not simply block the request; instead, it activates the transformation pipeline. This involves invoking other modules within the `safety` directory to analyze the prompt's intent and semantically replace harmful concepts with safe, artistic equivalents. The filtered and transformed prompt is then passed on to one of the art generation modules (e.g., `art/descriptive/main.lua`). This redirection is the essence of "flood-routing": channeling dangerous content away from its intended destructive path and into a constructive, compliant artistic output. This design is analogous to security interceptors in web applications, which handle cross-cutting concerns like auditing and security before a request reaches its main handler [103], or the plugins used in modern API gateways like Apache APISIX to enforce dynamic security policies [151][153].

To power the analytical capabilities of the filter, several supporting modules are required. The `safety/policy_loader.lua` module is responsible for managing the rules of engagement. It would load and maintain lists of prohibited words, phrases, and concepts, as well as higher-level policy guidelines. These could be sourced from a JSON or YAML file, allowing administrators to update the policy database without changing the core code. This module would provide an interface for the `filter.lua` to query whether a given term or phrase violates policy. Building on this, the `safety/llm_classifier.lua` module introduces a layer of semantic understanding. Instead of relying solely on keyword matching, this module would integrate with a Large

Language Model (LLM) fine-tuned for policy alignment and content moderation [74] [126]. Such models have been optimized to identify which policy specifications are relevant to a given piece of text, offering a much more nuanced form of analysis than simple pattern matching [30]. This classifier could assign a "danger score" to a prompt, helping the `filter.lua` make more informed decisions about whether a prompt is maliciously ambiguous or genuinely benign. The use of smaller, specialized LLMs for such tasks is a growing trend, balancing the need for accuracy with the constraints of computational cost and latency [93].

Once a prompt is flagged as potentially unsafe, the `safety/sanitizer.lua` module takes over the transformation process. This is arguably the most innovative part of the architecture. Its job is to take the original, harmful prompt and apply a series of rules or generative techniques to neutralize its negative content. For example, the word "fire" might be replaced with "a mesmerizing dance of light and shadow," and "violence" could be transformed into "dynamic movement and energy." This process could involve a hybrid approach: a set of predefined substitution rules for common trigger words, combined with a generative model trained to produce abstract or metaphorical representations of dangerous concepts [33]. Research into red-teaming text-to-image models highlights the need for such filters to be robust against adversarial prompts designed to bypass them [100] [112]. The sanitizer's output is a new, sanitized prompt that retains the *subject* of the original request but discards its harmful *intent*. Finally, the `safety/route_manager.lua` acts as a dispatcher. After the sanitizer has done its work, this module decides which artistic generation module is best suited to render the new, compliant concept. If the sanitized prompt is still about a battlefield, it might choose the `art/knowledge_graph` module to create an abstract representation of strategic points. If it's about a stormy sea, it might be routed to the `art/handpainting` module to generate a Van Gogh-style painting. This intelligent routing ensures that every intercepted request finds a creative and safe outlet.

This comprehensive safety architecture must also incorporate the initial age-gating control. As specified, this would be a separate mechanism, entirely independent of the chat platform itself but activated when a user requests image generation . The simplest implementation would be a check in the top-level `main.lua` or a dedicated `age_gate.lua` module. Before any request is passed to the safety filter, it would verify the user's age. If the user is deemed underage, all requests for image generation would be blocked or rerouted to a pre-approved list of safe, non-violent templates. This provides a clear, binary enforcement of the age restriction at the earliest possible stage of the request pipeline. The combination of a coarse-grained age gate followed by a fine-grained, multi-layered "flood-routing" filter creates a robust, defense-in-depth safety

system. It not only prevents harmful content from being generated but also actively seeks to transform potentially harmful user intent into a positive, creative expression, fully realizing the ambitious goals set forth for PromptLang-Lua.

Module	Responsibility	Key Functionality
`safety/filter.lua`	Central interception and routing hub.	Activates on triggers; invokes classifiers and sanitizers; routes to art modules. [103][151]
`safety/policy_loader.lua`	Manages the database of prohibited terms and policies.	Loads and provides access to rule sets for the filter.
`safety/llm_classifier.lua`	Performs semantic analysis of prompts for policy violations.	Integrates an LLM to assess intent and assign danger scores. [30] [74] [126]
`safety/sanitizer.lua`	Transforms detected harmful concepts into compliant artistic forms.	Applies rules and/or generative models to neutralize negative intent. [33] [99]
`safety/route_manager.lua`	Dispatches sanitized prompts to the appropriate generative art module.	Decides which art engine is best for the transformed concept.

Plugin Interface and Cross-Platform Compatibility

For PromptLang-Lua to fulfill its mission as a public-use, platform-agnostic plugin, its architecture must prioritize interoperability above all else . This means the system must be designed as a self-contained, externally manageable unit that can be easily dropped into a variety of host environments, from Discord bots to custom-built chat clients. The architectural blueprint for achieving this is heavily influenced by successful plugin systems in the wild, particularly Apache APISIX, which uses Lua for its powerful, high-performance plugin ecosystem [151][153]. The core of this compatibility strategy lies in abstraction: creating a clean, consistent boundary between the host platform and the PromptLang-Lua engine. This is achieved through a dedicated `platform` directory containing a thin abstraction layer and a manifest file that describes the plugin's capabilities. This design ensures that changes in the host platform do not ripple through the core logic of PromptLang-Lua, and conversely, updates to the assistant do not require modifications to the host application.

The cornerstone of the plugin interface is the `platform/chat_interface.lua` file. This module is responsible for abstracting away all the specific details of the host chat platform. Every chat platform—Discord, Slack, Telegram, etc.—has its own unique API for sending messages, receiving user input, managing sessions, and triggering actions. Instead of hardcoding calls to any one of these APIs within the core logic of PromptLang-Lua, the `chat_interface.lua` provides a unified set of wrapper functions. For

example, it would export a function like `send_message(recipient_id, message_content)` that internally translates `recipient_id` and `message_content` into the specific payload required by the target platform's API. Similarly, it would handle event listeners for incoming messages, translating events from the platform's webhook or event stream into a standardized format that the main logic in `main.lua` can understand. This abstraction layer is the key to cross-platform compatibility; to add support for a new platform, a developer would only need to write the platform-specific implementations inside `chat_interface.lua`, without ever touching the generative art or safety modules. This design pattern is a classic solution for managing cross-cutting concerns and ensuring that the core business logic remains pure and platform-independent [103]. The success of Lua as a scripting language for extending applications like Pandoc underscores its suitability for such embedding and extension scenarios [61].

Complementing the abstract interface is the `platform/plugin_manifest.json` file. This file is a static JSON document that serves as a machine-readable and human-readable description of the plugin. It is a critical component for discoverability and integration. The manifest should contain essential metadata such as the plugin's name (`PromptLang-Lua`), a version number (e.g., `1.0.0`), a short description of its capabilities, and a list of its dependencies (e.g., specific Lua libraries or external services). Crucially, it should also declare the functions and commands that the plugin exposes to the host platform. For instance, it might specify that the plugin responds to the command `!image` and supports the generation of 'descriptive', 'graph', and 'handpainting' styles. This manifest acts as a contract between PromptLang-Lua and the host application. When a platform's marketplace or integration system scans for available plugins, it can read this manifest to understand what the plugin does and how to configure it, dramatically lowering the barrier to entry for developers and end-users alike. This approach is analogous to the package manifests used in Linux distributions or Python's PyPI, which provide a standardized way to describe software packages [131][132].

Designing the entire system as a cohesive plugin also involves considerations of size, performance, and resource management. The LuaJIT engine, often used with Lua, is favored in high-performance contexts like APISIX because of its speed [153]. Furthermore, the practice of selectively opening only the necessary standard Lua libraries, as detailed in the official documentation, is a direct strategy to minimize the memory footprint and startup time of the plugin [121]. This is vital for a system intended for free, public use, as it ensures that integrating PromptLang-Lua does not impose a significant performance penalty on the host application. The modular structure itself contributes to this efficiency; a host platform that only needs the descriptive art feature does not need to load the

entire knowledge graph generation module, although in a typical Lua implementation, all required files would be loaded at startup. However, the clean separation allows for future optimization, such as lazy-loading modules only when their specific functionality is requested.

Finally, comprehensive documentation is a non-negotiable part of being a "free, public-use" plugin. The inclusion of a `docs/README.md` file is essential. This file should serve as the primary onboarding resource, providing clear instructions for installation, configuration, and usage. It should include examples of how to use the bracketed escape-sequences, demonstrate the different artistic styles, and explain how to set up the safety and age-gating features. It should also contain API documentation for developers who might want to extend the plugin or integrate it deeply into their own systems. Good documentation is a key factor in the adoption and long-term success of any open-source project. By combining a clean abstraction layer, a descriptive manifest file, a focus on performance, and excellent documentation, the PromptLang-Lua architecture can successfully meet the challenge of being a truly versatile and widely usable AI art assistant plugin.

Component	Location	Purpose	Key Characteristics
Abstraction Layer	`platform/chat_interface.lua`	Translates host platform APIs into a unified interface for the core logic.	Enables cross-platform compatibility by isolating platform-specific code. [103][153]
Plugin Manifest	`platform/plugin_manifest.json`	Describes the plugin's capabilities, version, and dependencies for discovery.	Standardized metadata for easy integration and management. [131][132]
Performance Optimization	Selective library loading, LuaJIT usage.	Minimizes memory footprint and maximizes execution speed.	Follows best practices for creating lean Lua applications. [121][153]
Documentation	`docs/README.md`	Provides setup, usage, and API instructions for users and developers.	Essential for public adoption and ease of use.

Syntax and Expression Handling: A Custom Parser for Domain-Specific Language

A defining feature of PromptLang-Lua is its reliance on syntactically accurate bracketed expressions, referred to as "escape-sequences," to control its various functions . This requirement moves the system beyond standard text processing and into the domain of a Domain-Specific Language (DSL). A DSL is a specialized computer language tailored to a particular problem domain, and building one correctly necessitates a dedicated parser. The optimal approach for PromptLang-Lua is to implement a custom recursive descent

parser, a technique that is well-suited to the simplicity and power of the Lua language [67] [68]. This parser would be encapsulated in its own module, `lib/parser.lua`, making it a reusable and testable component of the larger system. The design of this parser is foundational, as it dictates the grammar of the PromptLang-Lua language and directly influences how users will interact with its advanced features.

The choice of a recursive descent parser is strongly supported by the nature of the Lua language itself. Lua's own parser is a classic example of a recursive descent parser, known for being simple and effective for small languages and DSLs, as opposed to the complex parsers used by languages like C++ or Java [66] [68]. A recursive descent parser works by writing a set of mutually recursive procedures, where each procedure corresponds to a rule in the language's grammar [67]. For PromptLang-Lua, this means defining procedures for parsing different types of bracketed expressions, such as directives (`{cmd:generate, style:impressionist}`), data structures (`[item1, item2]`), or inline commands. The ability to handle nested expressions, such as `((3 == 5 AND 4 == ...))`, is a hallmark of this parsing technique and is directly applicable to the complex, hierarchical nature of DSL syntax [83] [104]. While Lua's built-in string patterns are powerful, they are fundamentally regular expressions and are not sufficient for parsing context-free grammars, which is why a custom implementation is necessary [18]. The challenge lies in defining a clear and unambiguous grammar for the bracketed expressions and then meticulously coding the parser to recognize it.

The `lib/parser.lua` module would be responsible for taking a raw string input from the user and producing a structured Abstract Syntax Tree (AST) or an equivalent data structure. This AST represents the parsed input in a way that is easy for the subsequent stages of the application to interpret. For example, the input `{set:style, value:[impasto, vibrant], target:sky}` would be parsed into a tree-like structure with a root node of type "command", having a property "name" with the value "set", and a property "arguments" containing another structure representing the key-value pairs. This structured output is far more powerful than the original string because it carries explicit semantic meaning. The safety filter can then traverse this tree to find commands related to image generation, the art modules can extract styling instructions, and the renderer can understand how to format the final output. This separation of parsing (syntactic analysis) from interpretation (semantic analysis) is a fundamental principle of compiler and interpreter design [66]. Resources on building DSL parsers and rules engines highlight the importance of this distinction, especially when dealing with complex expressions that need normalization [20].

Implementing such a parser requires careful attention to detail and a solid understanding of parsing theory. The process begins with defining the formal grammar of the PromptLang-Lua language. This grammar would specify the valid combinations of symbols, keywords, and bracket types. For instance, it might define a `<directive>` as a left curly brace, followed by a comma-separated list of `key:value` pairs, followed by a right curly brace. Once the grammar is defined, each non-terminal symbol in the grammar becomes a function in the `parser.lua` module. For example, there would be a `parse_directive()` function that implements the grammar rule for directives. These functions would call each other recursively to build up the parse tree. The process of building a parser for nested parentheses is a common exercise in compiler courses and demonstrates the recursive nature of the technique [83]. Libraries and frameworks exist for building parsers in various languages, including recursive descent parser generators for Python and JavaScript, which can serve as conceptual models [138][139]. Although Lua may not have a similarly mature ecosystem of off-the-shelf parser generators, its simplicity makes writing a parser by hand straightforward and efficient.

Beyond basic parsing, the `lib/parser.lua` module can also incorporate rudimentary validation. As it builds the AST, it can check for common syntax errors, such as unmatched brackets or invalid characters, and return meaningful error messages to the user. This immediate feedback is crucial for usability. For example, if a user types `{invalid syntax}`, the parser can report "Error on line 1: Expected a key-value pair after '{'." This level of detail helps users learn the language's syntax more quickly. The ability to accurately capture and represent the structure of expressions is also relevant in other domains, such as monitoring hard real-time distributed systems, where precise grammar-based parsing is essential [61]. Ultimately, the custom parser is the linchpin of PromptLang-Lua's identity. It transforms the system from a simple AI assistant into a programmable tool, empowering users to exert fine-grained control over the artistic generation process through a well-defined and syntactically rigorous language.

Synthesis and Strategic Recommendations for Implementation

The comprehensive research and analysis culminate in a coherent and actionable architectural blueprint for PromptLang-Lua. The optimal structure is a modular, yet tightly integrated, system comprising up to twenty distinct files, organized to reflect clear separation of concerns while enabling robust, cross-functional collaboration, particularly

for the critical safety mechanisms. The proposed architecture balances the user's demand for both deep integration for "flood-routing" and broad interoperability as a platform-agnostic plugin. This is achieved by designing the system as a collection of loosely coupled modules that communicate through well-defined interfaces and a central orchestrator. The following synthesis consolidates the key findings and provides a set of strategic recommendations for the phased implementation of this architecture.

The proposed file structure is designed for clarity, scalability, and maintainability. It is divided into four primary functional areas: 1. **Core System & Initialization (`main.lua`, `config/`, `lib/utils.lua`):** This layer provides the foundational infrastructure, managing the Lua runtime, loading configurations, and offering a standardized API for host platforms. 2. **Generative Art Engine (`art/` directory):** This is the creative heart of the system, split into distinct modules for descriptive art, knowledge graph generation, and handpainting, each with its own orchestration and specialization files. 3. **Safety & Policy Compliance (`safety/` directory):** This is the system's conscience, implementing the "flood-routing" interception framework through a chain of modules for filtering, classification, sanitization, and intelligent routing. 4. **Plugin Interface (`platform/` directory):** This layer ensures portability and ease of integration by abstracting platform-specific details and providing a manifest for discovery.

The following table summarizes the complete proposed file structure:

Path	Purpose	Key Responsibilities
`main.lua`	Core Orchestrator	Initializes Lua state, loads modules, defines public API. [121][153]
`config/default.lua`	Default Configurations	Stores fallback values for all configurable parameters. [29]
`config/schema.lua`	Configuration Schema	Defines the expected format for config files to ensure validity. [135]
`lib/utils.lua`	Utility Functions	Provides reusable code for string/table manipulation, logging, etc.
`lib/parser.lua`	DSL Parser	Implements a recursive descent parser for PromptLang-Lua's syntax. [67] [68]
`art/descriptive/main.lua`	Descriptive Art Orchestration	Manages prompts and interactions with text-to-image models. [11]
`art/descriptive/prompt_builder.lua`	Prompt Refinement	Structures and optimizes prompts for generation. [12]
`art/knowledge_graph/main.lua`	Knowledge Graph Generation	Accepts input and initiates graph construction. [62] [159]
`art/knowledge_graph/model_loader.lua`	Model Management	Handles loading and interacting with graph generation models. [125]
`art/handpainting/main.lua`	Handpainting Orchestration	Manages style transfer and generative painting. [14] [17]
`art/handpainting/style_transfer.lua`	Style Application	Applies learned artistic styles to subjects. [15]
`art/renderer.lua`	Output Formatting	Converts generated data into chat-displayable formats.
`safety/filter.lua`	Safety Interceptor	Central hub for "flood-routing" logic. [103][151]
`safety/policy_loader.lua`	Policy Database	Manages lists of prohibited terms and concepts.
`safety/llm_classifier.lua`	Semantic Analysis	Uses an LLM to assess prompt intent and danger. [74] [126]
`safety/sanitizer.lua`	Content Transformation	Replaces harmful concepts with compliant artistic ideas. [33] [99]
`safety/route_manager.lua`	Post-Sanitize Routing	Directs sanitized prompts to the appropriate art module.
`platform/chat_interface.lua`	Platform Abstraction	Hides host platform API differences behind a unified interface. [103]
`platform/plugin_manifest.json`	Plugin Metadata	Describes the plugin for discovery and integration. [131]
`docs/README.md`	Project Documentation	Onboarding guide for users and developers.

Based on this blueprint, the following strategic recommendations are proposed for a successful implementation:

First, **prioritize modularity and separation of concerns**. Strictly adhere to the file structure outlined above. Each module must have a single, well-defined responsibility.

This discipline is paramount for long-term maintainability and for preventing the system from becoming an unmaintainable "spaghetti" codebase. This approach is validated by microservice thinking applied to prompt engineering and modular agent architectures [38] [39].

Second, **implement the parser as the foundational milestone**. The `lib/parser.lua` module is the bedrock of the entire system's unique functionality. Developing this early, and defining the DSL grammar meticulously, will provide a concrete specification that guides the design and implementation of all other components. Without a working parser, the rest of the system cannot correctly interpret user commands.

Third, **adopt an incremental approach to the safety filter**. Begin with a minimal viable product for the safety module. Implement the `safety/filter.lua` and `safety/policy_loader.lua` with a robust keyword-matching system. This provides immediate value and establishes a baseline for reliability. The more computationally expensive `safety/llm_classifier.lua` and the complex `safety/sanitizer.lua` can be developed and integrated in subsequent phases, enhancing the system's intelligence over time.

Fourth, **design the entire project with a "Lua Library" mindset**. The final output should feel less like a standalone application and more like a high-quality, reusable library for the Lua ecosystem. This means focusing on clean function signatures, comprehensive error handling, and minimizing global state. Emulating the practice of selectively loading standard libraries can help keep the plugin's footprint lean and efficient [121].

By following this evidence-based architectural plan and these strategic recommendations, the development of PromptLang-Lua can proceed in a structured and efficient manner. This approach directly addresses all facets of the user's research goal, resulting in a system that is not only artistically capable and technically sound but also exceptionally safe, secure, and ready for widespread adoption across the AI chat landscape.

Reference

1. Generative AI for visualization: State of the art and future directions https://www.sciencedirect.com/science/article/pii/S2468502X24000160

2. From Pixels to Insights: Unsupervised Knowledge Graph Generation ... https://www.researchgate.net/publication/391044959_From_Pixels_to_Insights_Unsupervised_Knowledge_Graph_Generation_with_Large_Language_Model
3. [PDF] Multi-Modal Knowledge Graph Construction and Application: A Survey https://arxiv.org/pdf/2202.05786
4. A Survey of Multi-modal Knowledge Graphs: Technologies and Trends https://dl.acm.org/doi/10.1145/3656579
5. What Is a Knowledge Graph? | IBM https://www.ibm.com/think/topics/knowledge-graph
6. Bootstrapping Knowledge Graphs From Images and Text - Frontiers https://www.frontiersin.org/journals/neurorobotics/articles/10.3389/fnbot.2019.00093/full
7. [PDF] Scene Graph to Image Synthesis via Knowledge Consensus https://ojs.aaai.org/index.php/AAAI/article/view/25387/25159
8. [PDF] Prompt-Driven Development with Claude Code - arXiv.org https://arxiv.org/pdf/2601.17584
9. Prompt-Driven Development with Claude Code: Developing a TUI ... https://www.mdpi.com/2079-9292/15/4/903
10. 一文搞懂Prompt Engineering - wuxinhua.com https://wuxinhua.com/posts/prompt-engineering/
11. Understanding Prompt Structure: Key Parts of a Prompt https://learnprompting.org/docs/basics/prompt_structure?srsltid=AfmBOoqYrAXFxfLiL3SAUiq5zLp04xxdoBMRVTHb0pXpiLMe3mSUS4xp
12. [PDF] PromptPaint: Steering Text-to-Image Generation Through Paint ... https://arxiv.org/pdf/2308.05184
13. A Comparative Study of AI-Generated and Hand-Crafted Recreations https://dl.acm.org/doi/10.1145/3706598.3714157
14. Learning to paint artworks using stroke-style generative adversarial ... https://www.researchgate.net/publication/369176331_Stroke-GAN_Painter_Learning_to_paint_artworks_using_stroke-style_generative_adversarial_networks
15. A novel flexible identity-net with diffusion models for painting-style ... https://www.nature.com/articles/s41598-025-12434-4
16. Research on Emotion-Based Inspiration Mechanism in Art Creation ... https://www.mdpi.com/2227-7390/13/16/2597
17. [PDF] Ai–Powered Virtual Painting Platform - Atlantis Press https://www.atlantis-press.com/article/126017376.pdf

18. Optional capture of balanced brackets in Lua - Stack Overflow https://stackoverflow.com/questions/20636937/optional-capture-of-balanced-brackets-in-lua
19. Java balanced expressions check {[()]} - Stack Overflow https://stackoverflow.com/questions/23187539/java-balanced-expressions-check?page=2&tab=scoredesc
20. Creating a DSL expressions parser / rules engine - Stack Overflow https://stackoverflow.com/questions/62643102/creating-a-dsl-expressions-parser-rules-engine
21. Implementation, Integration, and Application of Embedded Domain ... https://search.proquest.com/openview/96ca48c9a647b88624bddea220d69c19/1?pq-origsite=gscholar&cbl=18750&diss=y
22. Programming Language Design Overview | PDF | Computer ... - Scribd https://www.scribd.com/document/856231414/Programming-Language-Design-and-Implementation
23. [PDF] Cisco ASA Series VPN ASDM Configuration Guide, 7.12 https://www.cisco.com/c/en/us/td/docs/security/asa/asa912/asdm712/vpn/asdm-712-vpn-config.pdf
24. Simple Index - NJU Mirror https://mirror.nju.edu.cn/pypi/web/simple
25. [PDF] arXiv:2402.18667v1 [cs.CL] 28 Feb 2024 https://arxiv.org/pdf/2402.18667
26. CHCS BK C945F7D2 00 Cisco Hcs Onboarding Guide 10 6 1 - Scribd https://www.scribd.com/document/449967861/CHCS-BK-C945F7D2-00-cisco-hcs-onboarding-guide-10-6-1
27. Arxiv今日论文| 2026-01-08 - 闲记算法 http://lonepatient.top/2026/01/08/arxiv_papers_2026-01-08
28. 从Prompt到Pipeline：构建企业级AI应用的架构思考 - CSDN博客 https://blog.csdn.net/qq_41187124/article/details/151118391
29. Prompt Engineering Pipeline Management to Production ... - LinkedIn https://www.linkedin.com/pulse/prompt-engineering-pipeline-management-production-lucas-rom%C3%A3o-l3xif
30. Aligning Generative AI for Regulatory and Operational Compliance https://arxiv.org/html/2503.04736v1
31. Develop advanced generative AI chat-based assistants by using ... https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/develop-advanced-generative-ai-chat-based-assistants-by-using-rag-and-react-prompting.html
32. A Generative AI Framework for Enhancing Human-AI Collaborative ... https://www.sciencedirect.com/org/science/article/pii/S1935568826000059
33. (PDF) Design Transformation Pathways for AI-Generated Images in ... https://www.researchgate.net/publication/

387980388_Design_Transformation_Pathways_for_AI-Generated_Images_in_Chinese_Traditional_Architecture

34. [PDF] Diffusion Self-Distillation for Zero-Shot Customized Image Generation https://openaccess.thecvf.com/content/CVPR2025/papers/Cai_Diffusion_Self-Distillation_for_Zero-Shot_Customized_Image_Generation_CVPR_2025_paper.pdf
35. Arxiv今日论文| 2026-03-04 - 闲记算法 http://lonepatient.top/2026/03/04/arxiv_papers_2026-03-04.html
36. A Safety Report on GPT-5.2, Gemini 3 Pro, Qwen3-VL, Grok 4.1 Fast ... https://arxiv.org/html/2601.10527v2
37. Learn AI agent delegation for workflow automation - LinkedIn https://www.linkedin.com/posts/alliekmiller_aiinwork-activity-7378831786199420928-7Nya
38. 从单体到模块化：Prompt-Engineering-Guide中的微服务思维实践 https://blog.csdn.net/gitblog_00119/article/details/151819129
39. Multi-Agent Collaboration for Automated Design Exploration on High ... https://arxiv.org/html/2603.11515v1
40. PromptScript：轻量级DSL 脚本，加速多样化的LLM 测试与验证 https://cloud.tencent.cn/developer/article/2349421
41. (PDF) A framework for assessing the capabilities of code generation ... https://www.researchgate.net/publication/401601040_A_framework_for_assessing_the_capabilities_of_code_generation_of_constraint_domain-specific_languages_with_large_language_models
42. tactus - PyPI https://pypi.org/project/tactus/0.44.0/
43. How AI can improve no-code tools for software platforms - LinkedIn https://www.linkedin.com/posts/ricky-ho-0ba168_in-many-software-platforms-we-offer-two-activity-7350306145305055233-0jCm
44. Cognos Report Studio: Cascading Prompts populating really slow https://stackoverflow.com/questions/21449536/cognos-report-studio-cascading-prompts-populating-really-slow
45. CotEditor 앱 - App Store https://apps.apple.com/us/app/coteditor/id1024640650?l=ko
46. (PDF) Automatic, Expressive, and Scalable Fuzzing with Stitching https://www.researchgate.net/publication/401132973_Automatic_Expressive_and_Scalable_Fuzzing_with_Stitching
47. Simple Index https://mirrors.aliyun.com/pypi/simple/
48. Release Notes — Airflow Documentation - Apache Airflow https://airflow.apache.org/docs/apache-airflow/2.7.2/release_notes.html

49. 插件8：拼写检查转载 - CSDN博客 https://blog.csdn.net/dearbaba_1666/article/details/80829311
50. A Review of Large Language Models Across Academic Disciplines https://arxiv.org/html/2509.19580v5
51. Why is GUI code so computationally expensive? - Stack Overflow https://stackoverflow.com/questions/502483/why-is-gui-code-so-computationally-expensive
52. [PDF] Flexible Self-Service Applications Using Process Choreography on ... https://www.redbooks.ibm.com/redbooks/pdfs/sg246356.pdf
53. [PDF] Cisco Expressway Administrator Guide (X12.5.1) https://www.cisco.com/c/dam/en/us/td/docs/voice_ip_comm/expressway/admin_guide/Cisco-Expressway-Administrator-Guide-X12-5-1.pdf
54. How to build an AI agent with LLM and Notion | Alfred Lua posted on ... https://www.linkedin.com/posts/alfredlua_aiagent-activity-7317392693905575936-RGz3
55. Assessing deep learning methods in cis-regulatory motif finding ... https://academic.oup.com/bib/article/23/1/bbab374/6381249
56. numpy.ndarray size changed, may indicate binary incompatibility ... https://stackoverflow.com/questions/66060487/valueerror-numpy-ndarray-size-changed-may-indicate-binary-incompatibility-exp
57. A Survey on Large Language Models for Code Generation https://dl.acm.org/doi/10.1145/3747588
58. PromptPaint: Steering Text-to-Image Generation Through Paint ... https://www.researchgate.net/publication/375063226_PromptPaint_Steering_Text-to-Image_Generation_Through_Paint_Medium-like_Interactions
59. https://mirrors.aliyun.com/debian-ports/indices/ov... https://mirrors.aliyun.com/debian-ports/indices/override.unstable.main
60. [PDF] Computer Science Curricula 2013 - ACM https://www.acm.org/binaries/content/assets/education/cs2013_web_final.pdf
61. Software-Pakete in »sid«, Unterbereich doc - Debian -- Packages https://packages.debian.org/de/sid/doc/
62. From Pixels to Insights: Unsupervised Knowledge Graph Generation ... https://www.mdpi.com/2078-2489/16/5/335
63. [PDF] Visual Traffic Knowledge Graph Generation from Scene Images https://openaccess.thecvf.com/content/ICCV2023/papers/Guo_Visual_Traffic_Knowledge_Graph_Generation_from_Scene_Images_ICCV_2023_paper.pdf
64. ChartKG: A Knowledge-Graph-Based Representation for Chart Images https://arxiv.org/html/2410.09761v1

65. [PDF] A Hybrid Deep Learning and Knowledge Graph Approach for ... - HAL https://hal.science/hal-05298604v1/file/APPLSCI.pdf
66. Generate output along with recursive descent parser - Stack Overflow https://stackoverflow.com/questions/7831334/generate-output-along-with-recursive-descent-parser
67. （三十一）深度解析领域特定语言（DSL）第六章——语法分析 https://blog.csdn.net/weixin_46217641/article/details/148711810
68. Plotkin's ACM SIGPLAN Award Year | Compiler | Java Virtual Machine https://www.scribd.com/document/838948434/TalkingCompilersWithGPT
69. MyTimeMachine: Personalized Facial Age Transformation - arXiv https://arxiv.org/html/2411.14521v1
70. [PDF] hangzhou tongshifu cultural and creative (group) co., ltd. 杭州銅師傅 ... https://www1.hkexnews.hk/app/sehk/2025/107856/documents/sehk25111101605.pdf
71. [PDF] PromptCharm: Text-to-Image Generation through Multi-modal ... https://arxiv.org/pdf/2403.04014
72. [PDF] Cisco Expressway Administrator Guide (X12.5) https://www.cisco.com/c/dam/en/us/td/docs/voice_ip_comm/expressway/admin_guide/Cisco-Expressway-Administrator-Guide-X12-5.pdf
73. 李世龙/Awesome-Code-LLM https://gitee.com/lishilong625/Awesome-Code-LLM
74. Safety Mechanisms, Training Paradigms, and Emerging Challenges https://arxiv.org/html/2507.19672v1
75. [PDF] AI and Cities - UN-Habitat https://unhabitat.org/sites/default/files/2022/10/artificial_intelligence_and_cities_risks_applications_and_governance.pdf
76. Integration of Multi-Agent Systems and Artificial Intelligence in Self ... https://www.mdpi.com/2227-9717/13/4/1144
77. [PDF] Governing with Artificial Intelligence (EN) - OECD https://www.oecd.org/content/dam/oecd/en/publications/reports/2025/06/governing-with-artificial-intelligence_398fa287/795de142-en.pdf
78. Daytime Star in real life (Generated by AI) This manhwa is very cute ... https://www.tiktok.com/@myaimind/video/7266578507277274373
79. ChatGPT: How to use long texts of unknown content in a prompt? https://stackoverflow.com/questions/75777566/chatgpt-how-to-use-long-texts-of-unknown-content-in-a-prompt
80. Release Notes — Airflow 2.11.2 Documentation https://airflow.apache.org/docs/apache-airflow/2.11.2/release_notes.html

81. ANTLR: how to parse a region within matching brackets with a lexer https://stackoverflow.com/questions/6918555/antlr-how-to-parse-a-region-within-matching-brackets-with-a-lexer
82. Debian -- Software Packages in "bullseye" https://packages.debian.org/bullseye/amd64/allpackages
83. Recursive Descent Parser and Nested Parentheses - Stack Overflow https://stackoverflow.com/questions/11404482/recursive-descent-parser-and-nested-parentheses
84. [PDF] Stream Processing Using Grammars and Regular Expressions - arXiv https://arxiv.org/pdf/1704.08820
85. [PDF] The Architecture of Open Source Applications Amy Brown and Greg ... https://kolegite.com/EE_library/books_and_lectures/%D0%9F%D1%80%D0%BE%D0%B3%D1%80%D0%B0%D0%BC%D0%B8%D1%80%D0%B0%D0%BD%D0%B5/The%20Architecture%20Of%20Open%20Source%20Applications%20%28Amy%20Brown%20and%20Greg%20Wilson%20%28eds.%29%29%20%28z-lib.org%29.pdf
86. CAX: Cellular Automata Accelerated in JAX - arXiv https://arxiv.org/html/2410.02651v2
87. [PDF] NILM assisté par capteurs à l' aide d'algorithmes d'apprentissage ... https://theses.hal.science/tel-05264612v1/file/BALTI_Nidhal.pdf
88. iOS Metal. Why does simply changing colorPixelFormat result in ... https://stackoverflow.com/questions/54256198/ios-metal-why-does-simply-changing-colorpixelformat-result-in-brighter-imagery
89. A Mathematical Theory of Communication: | Guide books https://dl.acm.org/doi/10.5555/1102016
90. Knowledge Graph Embedding Based on Graph Neural Network https://www.researchgate.net/publication/372677431_Knowledge_Graph_Embedding_Based_on_Graph_Neural_Network
91. 003柯林斯词典推送c | PDF - Scribd https://www.scribd.com/document/929133375/003%E6%9F%AF%E6%9E%97%E6%96%AF%E8%AF%8D%E5%85%B8%E6%8E%A8%E9%80%81-c
92. Literature and Visual Technologies - Writing After Cinema - Springer https://link.springer.com/content/pdf/10.1057/9780230389991.pdf
93. [PDF] LLMs in Software Security: A Survey of Vulnerability Detection ... https://arxiv.org/pdf/2502.07049v2.pdf?ref=applied-gai-in-security.ghost.io
94. (PDF) The design of simpleITK - ResearchGate https://www.researchgate.net/publication/259701414_The_design_of_simpleITK

95. PhpStorm 2024.3 (243.21565.202 build) Release Notes https://youtrack.jetbrains.com/articles/WI-A-231736192/PhpStorm-2024.3-243.21565.202-build-Release-Notes
96. Python Programming Practice Problems - Scribd https://www.scribd.com/document/963423493/Python-Problems
97. ChangeLog.openSUSE-Leap-15.4-x86_64-aarch64 ... - Index of / https://ftp.sjtu.edu.cn/opensuse/distribution/leap/15.4/ChangeLogs/ChangeLog.openSUSE-Leap-15.4-x86_64-aarch64-ppc64le-s390x-Build157.1-Media1.txt
98. [PDF] Artificial-intelligence-machine-learning-and-deep ... - ResearchGate https://www.researchgate.net/profile/Bahman-Zohuri/publication/362029792_Artificial_intelligence_machine_learning_and_deep_learning_use_cases/links/62d6f4fd441ed55f843b6956/Artificial-intelligence-machine-learning-and-deep-learning-use-cases.pdf
99. Safe Text-to-Image Generation: Simply Sanitize the Prompt ... - arXiv https://arxiv.org/html/2411.10329v1
100. Automated Red-Teaming of Text-to-Image Models via LLM-Driven ... https://www.researchgate.net/publication/397006711_AutoPrompt_Automated_Red-Teaming_of_Text-to-Image_Models_via_LLM-Driven_Adversarial_Prompts
101. Beyond the Safety Tax: Mitigating Unsafe Text-to-Image Generation ... https://arxiv.org/html/2508.21099v3
102. Arxiv今日论文| 2026-01-05 - 闲记算法 http://lonepatient.top/2026/01/05/arxiv_papers_2026-01-05.html
103. Where and how to use interceptors in web application? https://stackoverflow.com/questions/7083559/where-and-how-to-use-interceptors-in-web-application
104. Equation (expression) parser with precedence? - Stack Overflow https://stackoverflow.com/questions/28256/equation-expression-parser-with-precedence
105. Why are iframes considered dangerous and a security risk? https://stackoverflow.com/questions/7289139/why-are-iframes-considered-dangerous-and-a-security-risk
106. words-333333 - cs.Princeton https://www.cs.princeton.edu/courses/archive/spring18/cos226/assignments/autocomplete/testing/words-333333.txt
107. Adventures in 3D Printing PDF - Scribd https://www.scribd.com/document/367128267/ADVENTURES-IN-3D-PRINTING-pdf
108. How to generate an image in python pixel by pixel? - Stack Overflow https://stackoverflow.com/questions/17334629/how-to-generate-an-image-in-python-pixel-by-pixel
109. Recent Progress in Automated Code Generation from GUI Images ... https://www.researchgate.net/publication/

346495868_Recent_Progress_in_Automated_Code_Generation_from_GUI_Images_Using_Machine_Learning_Techniques

110. Machine Learning in Python: Main Developments and Technology ... https://www.mdpi.com/2078-2489/11/4/193
111. CristalX: Facilitating simulations for experimentally obtained grain ... https://www.sciencedirect.com/science/article/pii/S2352711021000145
112. ART: Automatic Red-teaming for Text-to-Image Models to Protect ... https://arxiv.org/html/2405.19360v2
113. (PDF) ART: Automatic Red-teaming for Text-to-Image Models to ... https://www.researchgate.net/publication/381006713_ART_Automatic_Red-teaming_for_Text-to-Image_Models_to_Protect_Benign_Users
114. Drift Net: The Aesthetics of Literature and Media in Migration - jstor https://www.jstor.org/stable/pdf/10.3998/mpub.14606494.pdf
115. [PDF] Retracing the Black Venus - eScholarship https://escholarship.org/content/qt4z29231s/qt4z29231s.pdf
116. Fork of NLP Assignment - Kaggle https://www.kaggle.com/code/mihirprajapati01/fork-of-nlp-assignment
117. 人工智能2026_3_12 - arXiv每日学术速递 https://www.arxivdaily.com/thread/77613
118. Arxiv今日论文| 2026-01-19 - 闲记算法 http://lonepatient.top/2026/01/19/arxiv_papers_2026-01-19
119. Annual Meeting of the Association for Computational Linguistics ... https://aclanthology.org/events/acl-2025/
120. [PDF] 首届中英三校博士生联合论坛 暨2025年西交利物浦大学博士生论坛 https://www.xjtlu.edu.cn/wp-content/uploads/2025/04/2025-Symposium-Abstract-Brochure.pdf
121. Best way to omit Lua standard libraries? - Stack Overflow https://stackoverflow.com/questions/966162/best-way-to-omit-lua-standard-libraries
122. XuanJia: A Comprehensive Virtualization-Based Code Obfuscator ... https://arxiv.org/html/2601.10261v1
123. 333333 23135851162 the 13151942776 of 12997637966 ftp://ftp.cs.princeton.edu/pub/cs226/autocomplete/words-333333.txt
124. glove.6B.100d.txt-vocab.txt - CodaLab https://worksheets.codalab.org/rest/bundles/0xadf98bb30a99476ab56ebff3e462d4fa/contents/blob/glove.6B.100d.txt-vocab.txt
125. Comprehensive Review of Graph Neural Networks: Challenges ... https://onlinelibrary.wiley.com/doi/10.1111/exsy.70091
126. Construction of Legal Knowledge Graph Based on ... - MDPI https://www.mdpi.com/2078-2489/15/11/666

127. SmolDocling: An ultra-compact vision-language model for end-to ... https://arxiv.org/html/2503.11576v1

128. Type of graph cut algorithm for 3D reconstruction - Stack Overflow https://stackoverflow.com/questions/14736887/type-of-graph-cut-algorithm-for-3d-reconstruction

129. Pushing the Level of Abstraction of Digital System Design - ACM https://dl.acm.org/doi/fullHtml/10.1145/3532989

130. Cool Linux Hacks 2024 | PDF - Scribd https://www.scribd.com/document/765657147/Cool-Linux-Hacks-2024

131. compact compressed textlist - Debian -- Packages https://packages.debian.org/bullseye/allpackages?format=txt.gz

132. Compare Packages Between Distributions - DistroWatch.com https://distrowatch.com/dwres.php?resource=compare-packages&firstlist=biolinux&secondlist=debian&=0&secondversions=2&showall=yes

133. suse-sles-15-sp3-chost-byos-v20210827-hvm-ssd-x86_64 Package ... https://publiccloudimagechangeinfo.suse.com/amazon/suse-sles-15-sp3-chost-byos-v20210827-hvm-ssd-x86_64/package_changelogs.html

134. [PDF] Artificial Intelligence Index Report 2025 https://hai-production.s3.amazonaws.com/files/hai_ai_index_report_2025.pdf

135. A Survey on Large Language Model Benchmarks - arXiv.org https://arxiv.org/html/2508.15361v1

136. The Rise of Prompt Architecture - Dave Woodward - LinkedIn https://www.linkedin.com/pulse/rise-prompt-architecture-dave-woodward-p4slc

137. [PDF] Download the Watershed Analysis Manual https://dnr.wa.gov/sites/default/files/2025-03/bc_fpb_wsamanual_052011.pdf

138. Debian -- Software Packages in "trixie", Subsection python https://packages.debian.org/trixie/python/

139. IntelliJ IDEA 2024.3 (243.21565.193 build) Release Notes - YouTrack https://youtrack.jetbrains.com/articles/IDEA-A-2100662289/IntelliJ-IDEA-2024.3-243.21565.193-build-Release-Notes

140. [PDF] Cisco Packaged Contact Center Enterprise Features Guide ... https://www.cisco.com/c/en/us/td/docs/voice_ip_comm/cust_contact/contact_center/pcce/pcce_12_6_2/maintenance/guide/pcce_b_features-guide-1262.pdf

141. [PDF] METAGEN: A DSL, DATABASE, AND BENCHMARK ... - OpenReview https://openreview.net/pdf?id=Xh2MzYH1h9

142. dialogue between body and sound through multimodal interaction https://www.academia.edu/116277649/

DIALOGUE_BETWEEN_BODY_AND_SOUND_THROUGH_MULTIMODAL_INTERACTION_

143. Udemy For Business Course List | PDF | Sustainability - Scribd https://www.scribd.com/document/821166664/Udemy-for-Business-Course-List
144. Arxiv今日论文| 2026-01-06 - 闲记算法 http://lonepatient.top/2026/01/06/arxiv_papers_2026-01-06
145. 人工智能2026_2_20 - arXiv每日学术速递 https://arxivdaily.com/thread/76838
146. Appl. Sci., Volume 16, Issue 6 (March-2 2026) – 241 articles - MDPI https://www.mdpi.com/2076-3417/16/6
147. Adaptive Instructional Systems - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-031-92970-0.pdf
148. [XML] https://stackoverflow.com/sitemap-questions-72.xml https://stackoverflow.com/sitemap-questions-72.xml
149. HUSBAND, Timothy, The Wild Man. Medieval Myth and Symbolism ... https://www.scribd.com/document/448250387/HUSBAND-Timothy-The-Wild-Man-Medieval-Myth-and-Symbolism-pdf
150. Integration of extended reality technologies in transportation systems https://www.sciencedirect.com/science/article/pii/S2590123025014045
151. Code samples | Apache APISIX® -- Cloud-Native API Gateway and ... https://apisix.apache.org/docs/general/code-samples/
152. Wasm | Apache APISIX® -- Cloud-Native API Gateway and AI Gateway https://apisix.apache.org/docs/apisix/3.10/wasm/
153. Apache APISIX 3.0: 11 Highlights of Open Source API Gateway https://apisix.apache.org/blog/2022/11/02/apache-apisix-v3-preview/
154. workflow | Apache APISIX® -- Cloud-Native API Gateway and AI ... https://apisix.apache.org/docs/apisix/3.12/plugins/workflow/
155. adj_noun_pairs.txt https://cse.hkust.edu.hk/msbd5003/data/adj_noun_pairs.txt
156. Sample Outputs: openrouter/horizon-alpha - EQ-Bench https://eqbench.com/results/creative-writing-v3/openrouter__horizon-alpha.html
157. using Scene and Knowledge Graphs for HD-EPIC VQA Challenge https://arxiv.org/html/2506.08553v1
158. (PDF) Infrared and Visible Image Fusion Techniques Based on ... https://www.researchgate.net/publication/347677146_Infrared_and_Visible_Image_Fusion_Techniques_Based_on_Deep_Learning_A_Review
159. Open-Vocabulary Scene Graph Generation with Vision-Language ... https://arxiv.org/html/2404.00906v3

160. Frommer's Thailand With Your Family - Scribd https://www.scribd.com/document/63134968/Frommer-s-Thailand-With-Your-Family